AULA 9 · CONTINUIDADE DOS MVPS E ENTREGA FINAL

Do MVP apresentado ao projeto que continua no GitHub

O caderno final do curso. Um guia passo a passo para baixar o projeto, abrir no Antigravity, rodar, testar, corrigir e devolver pelo GitHub, com um pacote individual para cada um dos 21 MVPs.

Curso de Capacitação POLITEC/MT · Professor Renato Rosa · Dia 9 · Módulo 10 (Continuidade, GitHub e entrega final)

Sumario

ABERTURA

1 Boas-vindas à última etapa 4
2 O mapa do caderno 4
3 O contrato da continuidade 5
4 O estado atual dos 21 MVPs 5

PARTE 1 · GITHUB DO ZERO

5 A conta e o convite 7
6 Baixar o projeto: ZIP ou clone 7
7 Abrir a pasta no Antigravity 8
8 Rodar o app e os testes 9
9 Criar uma branch por tarefa 9
10 Fazer um commit que se entende 10
11 Enviar e abrir o Pull Request 10
12 Publicar no GitHub Pages 11
13 Usar issues para não perder o fio 11

PARTE 2 · ANTIGRAVITY E ANATOMIA DO APP

14 A anatomia de um app 13
15 O fluxo de uma alteração 13
16 Como pedir ao agente do Antigravity 14
17 O que cada teste verifica 15
18 Dados fictícios, backup e LGPD 15
19 Onde estão as pendências 15

PARTE 3 · OS 21 MVPS

20 Como usar os pacotes 17
21 Controle de Adiantamentos · Jaqueline e Linicker 17
22 Agendamento CIN · Elthon 19
23 Agiliza+ · Diego, Francineide, Andreia e Eric 22
24 Segunda via da CIN · André, Flávia e Glauce 24
25 Comissão de Ética · Letícia e Thiago 25
26 Controle do Processo de Aquisição · Kasteline 27
27 Custódia Definitiva · Edjane Ávila 29
28 DOCUMENTAPOLITEC · Aline Xavier 31
29 Mapa de Equipamentos · João Lucas 33
30 Gestão de Documentos · Vitória Araújo 35
31 NIHDE · Gestão de Identificados · Kesia Melo 37
32 Hórus · Banco de Demandas · Iohan 39
33 Inscrições em Cursos · Edilaura Rondon 41
34 Mapa de Distribuição · Igor Henrique 43
35 Metas pelo BI · Karine Assunção 45
36 Passivo Ambiental · Fábio 47
37 Renovação de Contratos · Rodrigo Neves 49
38 Registro de Não Conformidade · Sabrina Ellen 51
39 TAP Digital · Danielle, Cristiane e Luciana 52
40 TEIA · Triagem de E-mails · Nelson e Helen 54
41 TCIN · Trilha de Custódia · Silmara Bueno 56

PARTE 4 · RECEITAS DE INTEGRAÇÃO

42 E-mail e Apps Script 59
43 Google Sheets e CSV 59

44 OCR e PDF 60
45 Google Earth, KMZ e GeoJSON 60
46 QR Code e link de inscrição 61
47 Firestore e formulário público 61
48 Biometria e sistemas institucionais 62

PARTE 5 · EQUIPE, REVISÃO E ENTREGA

49 Papéis na continuidade 63
50 Revisão de código em 10 minutos 63
51 O aceite com o setor 64
52 O piloto e a linha de base 64
53 O que entregar no fim 65

PARTE 6 · FECHAMENTO

54 Checklist final do curso 66
55 Os erros que mais aparecem 66
56 Quiz final 67
57 Glossário 69
58 Referências e próximos passos 70

ABERTURA

Abertura

O que é o Caderno 9, como usar e o contrato da entrega.

1. Boas-vindas à última etapa
2. O mapa do caderno
3. O contrato da continuidade
4. O estado atual dos 21 MVPs

§ 1

Boas-vindas à última etapa

Este é o caderno que fecha o curso. Até aqui, cada grupo construiu um MVP funcional, com telas, dados fictícios, testes e um repositório no GitHub. A partir de agora, o trabalho continua com vocês: o professor entrega o projeto, e cada equipe assume a integração com o setor, o aceite e o piloto.

O caderno não ensina a programar do zero. Ele ensina a continuar um projeto que já funciona: baixar, abrir, entender, alterar, testar, versionar e devolver pelo GitHub.

A REGRA DE OURO

O `index.html` é gerado. Edite `src/`, `app.json`, `nucleo/`, `exemplos/` e `modelos/`. Depois rode `python construir.py`.

O QUE VOCÊ LEVA

21 pacotes individuais em `Aula 9/pacotes/`, um por app, com o repositório, o que falta, o passo a passo de integração, os comandos de teste e a mensagem de commit.

---

§ 2

O mapa do caderno

O caderno tem seis partes. A Parte 1 é o GitHub. A Parte 2 é o Antigravity e a anatomia de um app. A Parte 3 é o pacote dos 21 MVPs. A Parte 4 traz as receitas de integração. A Parte 5 é o trabalho em equipe. A Parte 6 é o aceite e a entrega final.

Parte	Pergunta que responde	Entregável
1. GitHub	Como baixar, versionar e devolver?	Repositório atualizado e Pull Request
2. Antigravity e app	Onde fica cada arquivo e como rodar?	App aberto e teste passando
3. Os 21 MVPs	O que falta no meu app?	Pacote individual do app
4. Integrações	Como ligar e-mail, Sheets, OCR e Earth?	Fila local e roteiro técnico
5. Equipe	Como trabalhar sem sobrescrever o outro?	Branch, issue, revisão e merge
6. Entrega	Como provar que está pronto?	Aceite, piloto e ata

TEMPO SUGERIDO

Leia a Parte 1 e a Parte 2 na aula. Use a Parte 3 como manual do seu app. As Partes 4 a 6 ficam para as duas semanas seguintes, com encontros de acompanhamento.

O material foi escrito para ser lido no computador, no tablet ou no celular. O HTML funciona offline. O PDF serve para imprimir e levar para a reunião com o setor.

---

§ 3

O contrato da continuidade

Cada grupo recebe um app que já funciona. O compromisso é não recomeçar do zero e não estragar o que já passa nos testes. O contrato tem cinco itens.

- [ ] O app continua abrindo offline e sem requisição de rede.
- [ ] O `index.html` continua sendo gerado pelo `construir.py`.
- [ ] Os dados fictícios continuam separados dos dados reais.
- [ ] Toda alteração entra por branch e Pull Request, com teste.
- [ ] O aceite e o piloto ficam registrados no repositório.

QUANDO

Use este contrato na primeira reunião com o setor.

POR QUE

Ele separa o que é demonstração do que é uso real.

EXEMPLO NA POLITEC

Um app que passa nos testes, mas usa CPF real no seed, não está pronto.

NÃO FAÇA

Não copie o `index.html` para editar à mão, não suba o ZIP com dados de produção e não faça commit de token, senha ou planilha com nome de servidor.

---

§ 4

O estado atual dos 21 MVPs

Os 21 apps ativos estão com o MVP funcional, a identidade visual, os dados fictícios e os testes. O painel marca 160 de 160 telas. O que falta não é tela: é integração, decisão do setor, aceite e piloto.

Indicador	Situação
Apps ativos	21
Telas planejadas	160 de 160
Estrutura	21 de 21 apps OK
Cenários de interface	133 de 133 OK
Feedback automatizável	Ondas 1 a 3 concluídas
O que falta	Integrações, aceite, piloto e decisões do setor

COMO LER OS PACOTES

Cada pacote tem a mesma estrutura. Comece pelo item **O que falta**. Se a sua tarefa não estiver ali, abra uma issue e combine com o professor.

PARTE 1

GitHub do zero

Conta, repositório, ZIP, clone, branch, commit, push e Pull Request.

1. A conta e o convite
2. Baixar o projeto: ZIP ou clone
3. Abrir a pasta no Antigravity
4. Rodar o app e os testes
5. Criar uma branch por tarefa
6. Fazer um commit que se entende
7. Enviar e abrir o Pull Request
8. Publicar no GitHub Pages
9. Usar issues para não perder o fio

§ 5

A conta e o convite

O GitHub é o lugar onde o projeto fica guardado e versionado. Cada app tem um repositório próprio. Para trabalhar, você precisa de uma conta e de um convite de colaborador.

1. Crie a conta em github.com com o e-mail que você usa no curso.
2. Confirme o e-mail. Sem confirmação, o GitHub não deixa você fazer push.
3. Envie seu login ao professor. O convite chega por e-mail e também aparece em **github.com/notifications**.
4. Aceite o convite. O repositório passa a aparecer na sua lista.
5. Se o convite não chegar, peça para o professor conferir a grafia do login.

SEM CONVITE?

Você ainda pode baixar o ZIP e estudar. Para enviar alterações sem convite, faça um **fork** e abra um Pull Request para o repositório original.

LOGIN E E-MAIL

Use o mesmo e-mail no GitHub e no computador. O Git identifica o autor do commit por `user.name` e `user.email`. Configure uma vez:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email"
```

§ 6

Baixar o projeto: ZIP ou clone

Há dois caminhos. O ZIP é mais simples para abrir e ler. O clone é o caminho certo para quem vai alterar e devolver.

ZIP

No repositório, clique em **Code** > **Download ZIP**. Extraia em uma pasta curta. O ZIP não traz o histórico do Git, então não serve para push direto.

Clone

No terminal, rode `git clone URL`. O clone traz o histórico, a branch e a ligação com o GitHub. É o caminho recomendado para a continuidade.

POWERSHELL

```
git clone https://github.com/renato0503/adiantamento.git
cd adiantamento
git status
git branch --show-current
```

ZIP E GIT

Se você baixou o ZIP e quer enviar alterações, não rode `git init` no meio da pasta antiga. Clone o repositório em uma pasta nova e copie para lá apenas os arquivos que você alterou.

---

§ 7

Abrir a pasta no Antigravity

O Antigravity trabalha com uma pasta aberta. A pasta precisa ser a raiz do app, aquela que tem o `app.json`, o `src` e o `construir.py`.

1. Abra o Antigravity.
2. Escolha **File** > **Open Folder**.
3. Selecione a pasta do app, não a pasta do curso inteiro.
4. Confira na barra lateral se aparecem `README.md`, `app.json`, `src`, `nucleo` e `construir.py`.
5. Abra o terminal integrado.
6. Peça ao agente para ler os arquivos antes de alterar qualquer coisa.

TEXTO

```
Você está no projeto <slug>. Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Antes de alterar, rode python construir.py e os testes.
Depois de alterar, mostre o diff e rode os testes de novo.
```

PASTA CERTA

Se o terminal mostrar a pasta do curso, com 21 apps dentro, você abriu a pasta errada. Abra a pasta do seu app.

§ 8

Rodar o app e os testes

O app é HTML, CSS e JavaScript. Para gerar o `index.html`, o projeto usa Python. Para os testes de interface, usa Playwright.

1. Confira o Python: `python --version`. Use 3.11 ou mais novo.
2. Rode `python construir.py` na raiz do app.
3. Abra o `index.html` no Edge ou no Chrome.
4. Vá em **Configuração** e clique em **Carregar dados fictícios**.
5. Rode `python testes/testar.py`.
6. Rode `python testes/cenarios.py`.

POWERSHELL

```
python --version
python construir.py
python testes/testar.py
python testes/cenarios.py
```

PLAYWRIGHT

Se o teste reclamar que o Playwright não está instalado, rode `pip install playwright` e `playwright install chromium`. O app em si não precisa de internet nem de instalação.

§ 9

Criar uma branch por tarefa

Nunca trabalhe direto na `main`. A branch isola a sua alteração e permite revisar antes de juntar.

1. Atualize a main: `git checkout main` e `git pull`.
2. Crie a branch: `git checkout -b melhoria/nome-curto`.
3. Faça uma tarefa por branch. Se misturar duas, a revisão fica difícil.
4. Rode os testes antes de abrir o Pull Request.
5. Envie: `git push origin melhoria/nome-curto`.

```powershell
git checkout main
git pull
git checkout -b melhoria/aviso-email
git status
```

NOME DA BRANCH

Use verbos curtos: `melhoria/aviso-email`, `correcao/prazo`, `integracao/sheets`. Evite acentos e espaços.

---

§ 10

Fazer um commit que se entende

Um commit é uma foto do projeto com uma mensagem. A mensagem precisa dizer o que mudou e por quê, não apenas "alterações".

1. Veja o que mudou: `git status` e `git diff`.
2. Adicione só o que pertence à tarefa: `git add arquivo`.
3. Escreva a mensagem no imperativo: `Adiciona aviso de vencimento`.
4. Faça o commit: `git commit -m "..."`.
5. Confira: `git log --oneline -3`.

```powershell
git status
git diff
git add src/app.js app.json
git commit -m "Adiciona aviso de vencimento ao fiscal"
git log --oneline -3
```

NUNCA COMMITE

Token, senha, arquivo `.env`, planilha com nome de servidor, foto de pessoa atendida ou backup real. Se commitou por engano, avise o professor antes de fazer push.

---

§ 11

Enviar e abrir o Pull Request

O push envia a branch para o GitHub. O Pull Request pede que alguém revise e junte a branch na main.

1. Envie a branch: `git push origin melhoria/nome-curto`.
2. Abra o repositório no navegador. O GitHub mostra o botão **Compare & pull request**.
3. Escreva o que mudou, o que testou e o que ainda falta.

4. Marque o professor ou um colega como revisor.
5. Depois do aceite, clique em **Merge pull request**.
6. Apague a branch remota depois do merge.

```powershell
git push origin melhoria/aviso-email
```

CONFLITO

Se o GitHub disser que há conflito, atualize a main, faça o merge dela na sua branch, resolva o arquivo, rode os testes e envie de novo.

---

§ 12

Publicar no GitHub Pages

O GitHub Pages serve o `index.html` como site. Para o app, ative na raiz do repositório, branch `main`.

1. Abra **Settings** > **Pages**.
2. Em **Source**, escolha **Deploy from a branch**.
3. Selecione `main` e a pasta `/ (root)`.
4. Salve e espere de 1 a 10 minutos.
5. Abra a URL publicada e teste o app e o link de inscrição.

CACHE

Se a página antiga continuar aparecendo, espere ou abra em uma janela anônima. O CDN do GitHub demora alguns minutos para atualizar.

---

§ 13

Usar issues para não perder o fio

Issue é uma tarefa ou um erro registrado. Ela guarda o print, o passo e a decisão, e evita que a informação fique só no WhatsApp.

1. Abra **Issues** > **New issue**.
2. No título, diga o app e o problema: `Inscrições: link não abre no Outlook`.
3. No corpo, coloque o passo, o esperado, o acontecido e o print.

4. Marque uma etiqueta: `bug`, `melhoria` ou `duvida`.
5. Feche a issue no commit ou no Pull Request que resolveu.

QUANDO

Use uma issue por pendência do pacote.

POR QUE

O pacote já lista o que falta; a issue transforma a lista em trabalho acompanhável.

EXEMPLO NA POLITEC

Se a integração com o e-mail vai levar duas semanas, a issue guarda o combinado e o responsável.

MODELO DE ISSUE

Título: **App: o que falta.** Corpo: **Passo**, **Esperado**, **Aconteceu**, **Ambiente** e **Print**.

PARTE 2

Antigravity e anatomia do app

Onde mexer, o que não mexer e como pedir ajuda ao agente.

1. A anatomia de um app
2. O fluxo de uma alteração
3. Como pedir ao agente do Antigravity
4. O que cada teste verifica
5. Dados fictícios, backup e LGPD
6. Onde estão as pendências

§ 14

A anatomia de um app

Todos os apps seguem a mesma arquitetura: um único HTML gerado, CSS e JavaScript embutidos, IndexedDB e dados fictícios. A pasta é pequena e tem sempre os mesmos arquivos.

Arquivo ou pasta	O que é	Você edita?
index.html	App gerado, com CSS, JS e dados embutidos	Não
app.json	Título, banco, perfis, rotas, libs e dados	Sim
src/app.js	Telas e regras das primeiras sprints	Sim
src/sprintN.js	Telas e regras das sprints seguintes	Sim
src/app.css	Ajustes visuais do app	Sim
nucleo/	Casca, roteador, IndexedDB, ícones e CSS comum	Com cuidado
exemplos/	Gerador e seed dos dados fictícios	Sim
modelos/	Modelos, roteiros e documentos de referência	Sim
vendor/	Bibliotecas de terceiros, com versão e SHA	Só ao trocar a lib
testes/	Testes de estrutura e cenários de interface	Sim

NUNCA EDITE O INDEX.HTML

Ele é sobrescrito pelo `construir.py`. Se você editar à mão, a alteração some no próximo build.

O `construir.py` lê o `app.json`, junta `src/*.js` em ordem e embute as bibliotecas e os dados. O resultado é um arquivo que abre com duplo clique.

§ 15

O fluxo de uma alteração

Toda alteração segue o mesmo caminho. Se você pular uma etapa, o erro aparece mais tarde e fica mais caro.



1. Leia a issue e o trecho do `sprint.md`.
2. Localize a tela e a regra em `src/`.
3. Altere uma coisa pequena.
4. Rode `python construir.py`.
5. Abra o app e reproduza o caso.
6. Rode os testes e confira o console.
7. Faça o commit e o pull request.

✗ ERRADO

Editar o index.html e mandar o print.

✓ CERTO

Editar a fonte, construir e rodar o teste.

UMA COISA POR VEZ

Se você mudar a tela, a regra e o seed no mesmo commit, fica impossível saber o que causou o erro.

§ 16

Como pedir ao agente do Antigravity

O agente é útil quando recebe contexto e um limite. Pedido vago gera alteração grande e difícil de revisar.

Pedido ruim

"Melhore o app". O agente não sabe o que falta, mexe em tudo e você não consegue revisar.

Pedido bom

"Leia o sprint.md. Na tela #/painel, adicione o botão Avisar. Não altere o index.html. Rode os testes e mostre o diff."

TEXTO

```
Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html.
Trabalhe apenas nesta tarefa: <descreva uma tarefa>.
Antes de alterar, rode python construir.py e os testes.
Depois de alterar, rode os testes de novo, mostre o diff e explique o que mudou.
```

ARQUIVOS DE CONTEXTO

Sempre peça para o agente ler `README.md`, `PREMISSAS.md`, `sprint.md` e o arquivo de `src/` que será alterado.

§ 17

O que cada teste verifica

O projeto tem dois níveis de teste. O primeiro confere a estrutura. O segundo usa a tela como uma pessoa usaria.

Comando	O que verifica	Quando rodar
python testes/testar.py	Zero rede, rotas, tema, dados fictícios, #/testes	Sempre
python testes/cenarios.py	Um cenário por sprint, com o “Pronto quando”	Antes do PR
python Apps/_base/testar.py	Todos os apps, no repositório do curso	Só no computador do professor
node testes/testar_code_gs.js	Servidor do agendamento CIN	Só no agendamento

TESTE QUE PASSOU NÃO É DESCULPA

O teste cobre o que foi escrito. Se você mudou uma regra, acrescente ou ajuste um cenário. Teste sem cenário novo não prova a regra nova.

Se o teste falhar, leia a primeira linha do erro. Corrija uma coisa, rode de novo. Não tente “consertar” o teste para ele passar se a regra está errada.

§ 18

Dados fictícios, backup e LGPD

Os apps usam dados fictícios. O uso real exige cuidado com dado pessoal, retenção e acesso.

- ☐ O seed não tem nome, CPF, foto ou endereço de pessoa real.
- ☐ O backup é feito em Configuração e guardado em local controlado.
- ☐ O app avisa a finalidade e o prazo de retenção.
- ☐ O perfil vê só o que precisa.
- ☐ A exclusão de um caso mantém apenas a contagem, quando a regra pede.

NUNCA

Não suba planilha real, print de tela com nome, backup de produção ou token para o repositório público. Se precisar testar, use o seed ou anonimize antes.

QUANDO

Use o botão de backup antes de cada reunião.

POR QUE

Se o navegador limpar os dados, o backup é a única cópia.

EXEMPLO NA POLITEC

O backup é um .json que pode ser restaurado em Configuração.

§ 19

Onde estão as pendências

Cada app tem três lugares para consultar o que falta. Leia nessa ordem.

Arquivo	O que traz	Como usar
README.md	Visão do projeto, time e links	Primeiro contato
sprint.md	Sprints, itens feitos e abertos	Lista de trabalho
PREMISSAS.md	O que foi assumido e o que confirmar	Reunião com o setor
modelos/roteiro-aceite.md	Roteiro de aceite e piloto	Antes da entrega
Ajuda Técnica no app	Passo a passo de integração externa	Quando for ligar serviço

AJUDA TÉCNICA

No rodapé do app, clique três vezes no nome do app. O painel de ajuda abre com o passo a passo do que depende de serviço externo. Ele não aparece no menu.

Se a pendência for uma decisão do setor, leve a pergunta escrita. Se for técnica, abra uma issue com o passo e o erro.

PARTE 3

Os 21 MVPs

Um pacote por app: o que está pronto, o que falta, como integrar e como devolver.

1. Como usar os pacotes
2. Controle de Adiantamentos · Jaqueline e Linicker
3. Agendamento CIN · Elthon
4. Agiliza+ · Diego, Francineide, Andreia e Eric
5. Segunda via da CIN · André, Flávia e Glauce
6. Comissão de Ética · Letícia e Thiago
7. Controle do Processo de Aquisição · Kasteline
8. Custódia Definitiva · Edjane Ávila
9. DOCUMENTAPOLITEC · Aline Xavier
10. Mapa de Equipamentos · João Lucas
11. Gestão de Documentos · Vitória Araújo
12. NIHDE · Gestão de Identificados · Kesia Melo
13. Hórus · Banco de Demandas · Iohan
14. Inscrições em Cursos · Edilaura Rondon
15. Mapa de Distribuição · Igor Henrique
16. Metas pelo BI · Karine Assunção
17. Passivo Ambiental · Fábio
18. Renovação de Contratos · Rodrigo Neves
19. Registro de Não Conformidade · Sabrina Ellen
20. TAP Digital · Danielle, Cristiane e Luciana
21. TEIA · Triagem de E-mails · Nelson e Helen
22. TCIN · Trilha de Custódia · Silmara Bueno

§ 20

Como usar os pacotes

Cada seção desta parte é um pacote. Ela traz o repositório, o que já está pronto, o que falta, o roteiro de integração, os comandos de teste e a mensagem de commit. O mesmo conteúdo está em `Aula 9/pacotes/<slug>.md`.

COMECE PELO SEU

Localize o app na lista, leia o pacote e abra a issue da primeira pendência. Não tente fazer todas de uma vez.

A ordem abaixo é a mesma do painel e do hub. Os links de repositório e de app publicado estão em cada seção.

---

§ 21

Controle de Adiantamentos · Jaqueline e Linicker

Equipe: Jaqueline e Linicker · **Tipo:** Aplicativo para servidor · **Repositório:** https://github.com/renato0503/adiantamento

Pedido de suprimento de fundos, três orçamentos, 12 etapas, urgência e recurso.

O QUE JÁ ESTÁ PRONTO

- Pedido com três orçamentos e CNPJ conferido
- Linha do tempo de 12 etapas, com devolução motivada
- Afastamento, urgência e recurso de 24 h
- Painel, checklist e dados fictícios

O QUE FALTA

- Validar as 12 etapas com a DAS e a contabilidade da SESP
- Confirmar a Orientação Técnica da CGE e os feriados
- Definir a origem dos afastamentos e a integração com o SICAD

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/adiantamento e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/adiantamento.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\adiantamento`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/adiantamento` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Afastamentos

1. Combine com o RH se a lista vem do SIGRH, de planilha ou de exportação manual.
2. Crie um importador de CSV em `#/config` e grave na loja `afastamentos`.
3. No pedido, bloqueie o servidor afastado e mostre a data de retorno.
4. Teste com um afastamento fictício e confira o bloqueio.

Aprovação e SICAD

1. Defina o canal de aprovação, e-mail ou fila local.
2. Registre o número SICAD no fechamento da etapa.
3. Inclua o número no CSV e na ficha do pedido.
4. Valide o fluxo com um pedido de teste, sem dado real.

Como testar

POWERSHELL

```
python construir.py
python testes/testar.py
python testes/cenarios.py
#/pedido/novo e #/pedido/:id
```

Prompt para colar no Antigravity

TEXTO

```
Você está no projeto Controle de Adiantamentos (adiantamento). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Validar as 12 etapas com a DAS e a contabilidade da SESP.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

TEXTO

```
Adiantamento: integra afastamentos e SICAD
```

Pacote individual: `Aula 9/pacotes/adiantamento.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/adiantamento/index.html.

---

§ 22

Agendamento CIN · Elthon

Equipe: Elthon · **Tipo:** Aplicativo web e Apps Script · **Repositório:** https://github.com/renato0503/agendamento-cin

Agenda pública dos postos de identificação civil, com vaga única e protocolo.

O QUE JÁ ESTÁ PRONTO

- Grade de horários e cadastro de postos
- Agendamento do cidadão com protocolo
- Painel do posto e comprovante
- Versão de demonstração offline

O QUE FALTA

- Publicar a agenda no Google Apps Script com a conta institucional
- Ligar `LockService`, `MailApp` e gatilho de lembrete
- Definir os postos do piloto, a grade real e as regras RN06 a RN10

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/agendamento-cin e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/agendamento-cin.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\agendamento-cin`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/agendamento-cin` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Publicar o Apps Script

1. Crie um projeto novo em script.google.com com a conta institucional.
2. Copie `Code.gs` e `index.html` do repositório para o projeto.
3. Em Implantar, escolha Aplicativo da Web, executar como você e acesso para qualquer pessoa com o link.
4. Autorize as permissões e copie a URL publicada.

Vaga única e e-mail

1. Use `LockService.getScriptLock()` em volta da gravação da reserva.
2. Grave cada vaga em uma linha da planilha, com chave posto, dia e hora.
3. Envie o comprovante com `MailApp.sendEmail`.
4. Crie um gatilho diário para lembrar o cidadão.

Teste com dois navegadores

1. Abra a agenda em duas janelas e tente reservar o mesmo horário.
2. Confirme que a segunda reserva recebe mensagem de vaga ocupada.
3. Confira o protocolo e o e-mail de teste.
4. Registre o resultado em `modelos/roteiro-aceite.md`.

Como testar

```powershell
node testes/testar_code_gs.js
python testes/testar.py
publique e abra a URL do Apps Script
```

Prompt para colar no Antigravity

```texto
Você está no projeto Agendamento CIN (agendamento-cin). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Publicar a agenda no Google Apps Script com a conta institucional.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```
TEXTO
Agendamento CIN: publica a agenda no Apps Script
```

Pacote individual: `Aula 9/pacotes/agendamento-cin.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/agendamento-cin/index.html.

---

§ 23

Agiliza+ · Diego, Francineide, Andreia e Eric

Equipe: Diego, Francineide, Andreia e Eric · **Tipo:** Automação de laudos · **Repositório:** https://github.com/renato0503/agiliza-mais

Gera o laudo de exame em celular a partir da pasta do caso e do modelo Word.

O QUE JÁ ESTÁ PRONTO

- Pasta do caso, extração e conferência
- Textos, quesitos e biblioteca
- Laudo `.docx` pelo modelo embutido
- Revisão, histórico e limpeza por LGPD

O QUE FALTA

- Ler a requisição em PDF com pdf.js e o OCR de imagem
- Receber o modelo institucional timbrado e os formatos reais dos TXT/HTML
- Fechar o Sprint 12 de aceite e o piloto

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/agiliza-mais e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/agiliza-mais.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\agiliza-mais`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/agiliza-mais` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.

7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

PDF e OCR

1. Ative o pdf.js já presente em `vendor/` e leia a requisição em `#/conferencia`.
2. Se o PDF não tiver texto, mostre a mensagem e permita digitar ou usar um OCR externo.
3. Registre a origem de cada campo lido.
4. Teste com uma requisição fictícia e confira o laudo.

Modelo institucional

1. Coloque o `.docx` em `modelos/` e os marcadores em `modelos/MARCADORES.md`.
2. Confira cada marcador no `docxtemplater`.
3. Gere um laudo de teste e abra no Word.
4. Peça a validação do perito antes do piloto.

Como testar

POWERSHELL

```
python construir.py
python testes/testar.py
python testes/cenarios.py
#/inicio, #/caso/:id e #/revisao
```

Prompt para colar no Antigravity

TEXTO

```
Você está no projeto Agiliza+ (agiliza-mais). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Ler a requisição em PDF com pdf.js e o OCR de imagem.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

TEXTO

```
Agiliza+: liga PDF, OCR e modelo institucional
```

Pacote individual: `Aula 9/pacotes/agiliza-mais.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/agiliza-mais/index.html.

§ 24

Segunda via da CIN · André, Flávia e Glauce

Equipe: André, Flávia e Glauce · **Tipo:** Aplicativo para cidadão · **Repositório:** https://github.com/renato0503/cin-segunda-via

Pedido de segunda via da Carteira de Identidade Nacional a distância, com conferência e fila.

O QUE JÁ ESTÁ PRONTO

- Pedido com elegibilidade e fotos de teste
- Quatro fotos de impressões digitais quando o documento foi extraviado
- Fila do servidor e protocolo
- Resumo para o sistema e troca de perfil

O QUE FALTA

- Integrar a base biométrica e o sistema de identificação
- Confirmar as regras oficiais da segunda via
- Validar a qualidade mínima das fotos e o fluxo de recusa

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/cin-segunda-via e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/cin-segunda-via.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\cin-segunda-via`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/cin-segunda-via` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Base biométrica

1. Peça à TI o contrato da API ou o caminho de exportação da base.
2. Crie uma fila local com o protocolo e o status da consulta.
3. Nunca grave a biometria no navegador; guarde só o resultado e o protocolo.
4. Teste com uma resposta simulada e registre no log.

Regras e aceite

1. Confirme quando a segunda via dispensa nova coleta.
2. Ajuste a mensagem de recusa e o prazo.
3. Rode o roteiro com a equipe de identificação.
4. Atualize `PREMISSAS.md` com as respostas.

Como testar

POWERSHELL

```
python construir.py
python testes/testar.py
python testes/cenarios.py
#/pedido e #/fila
```

Prompt para colar no Antigravity

TEXTO

```
Você está no projeto Segunda via da CIN (cin-segunda-via). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Integrar a base biométrica e o sistema de identificação.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

TEXTO

```
CIN 2ª via: prepara a integração biométrica
```

Pacote individual: `Aula 9/pacotes/cin-segunda-via.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/cin-segunda-via/index.html.

§ 25

Comissão de Ética · Letícia e Thiago

Equipe: Letícia e Thiago · **Tipo:** Automação de rotina · **Repositório:** https://github.com/renato0503/comissao-etica-denuncias

Registro, classificação, admissibilidade, oitivas, minutas e relatório da Comissão de Ética.

O QUE JÁ ESTÁ PRONTO

- Registro e classificação assistida com o artigo da LC 112/2002-MT
- Admissibilidade com membros e fundamento padrão
- Oitivas com exclusão e encerramento
- Minutas, relatório, encerramento e envio por e-mail

O QUE FALTA

- Receber os modelos oficiais de ata, oitiva e relatório
- Confirmar a lista nominal dos membros
- Definir o canal de e-mail institucional e as permissões de envio

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/comissao-etica-denuncias e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/comissao-etica-denuncias.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\comissao-etica-denuncias`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/comissao-etica-denuncias` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Membros e modelos

1. Troque a lista `CE_MEMBROS` em `src/sprint4.js` pelos nomes reais.
2. Coloque os modelos em `modelos/` e ajuste `CE_MINUTAS` em `src/sprint6.js`.
3. Valide a ata, o relatório e o encerramento com a Comissão.
4. Registre as diferenças em `PREMISSAS.md`.

Envio institucional

1. Decida se o envio será `mailto:` ou Apps Script.
2. Para Apps Script, crie uma função `enviarDocumento` e uma fila local.
3. Nunca coloque token no código; use as permissões da conta.
4. Teste com um documento sem dado real.

Como testar

POWERSHELL

```
python construir.py
python testes/testar.py
python testes/cenarios.py
#/caso/:id/documentos e #/relatorio
```

Prompt para colar no Antigravity

TEXTO

```
Você está no projeto Comissão de Ética (comissao-etica-denuncias). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Receber os modelos oficiais de ata, oitiva e relatório.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

TEXTO

```
Comissão de Ética: usa os modelos e o canal oficiais
```

Pacote individual: `Aula 9/pacotes/comissao-etica-denuncias.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/comissao-etica-denuncias/index.html.

---

§ 26

Controle do Processo de Aquisição · Kasteline

Equipe: Kasteline · **Tipo:** Painel de BI · **Repositório:** https://github.com/renato0503/controle-aquisicoes

Nove etapas da contratação, documentos, três cotações, alerta de preço e base legal.

O QUE JÁ ESTÁ PRONTO

- Fluxo de 9 etapas ancorado na Lei 14.133/2021
- Pesquisa de preços com três cotações e memória de cálculo
- Alerta de estimado x cotado e histórico por objeto
- Painel compacto e conferência no celular

O QUE FALTA

- Receber os modelos oficiais de DFD, ETP, TR e análise de riscos
- Confirmar prazos, permissões e a base de 2024/2025
- Validar o alerta de 30% com o setor

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/controle-aquisicoes e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/controle-aquisicoes.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\controle-aquisicoes`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/controle-aquisicoes` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Modelos oficiais

1. Salve os modelos em `modelos/` sem dado real.

2. Ajuste os campos e os anexos obrigatórios em `src/app.js`.
3. Faça um processo de teste do DFD ao parecer.
4. Confira a base legal exibida na ficha.

Base e permissões

1. Importe a base histórica em CSV e confira as colunas.
2. Defina quem cria, quem aprova e quem só consulta.
3. Teste cada perfil e registre no roteiro de aceite.
4. Meça o tempo do processo antes e depois.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/processos e #/precos
```

Prompt para colar no Antigravity

```texto
Você está no projeto Controle do Processo de Aquisição (controle-aquisicoes). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Receber os modelos oficiais de DFD, ETP, TR e análise de riscos.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```texto
Aquisições: integra modelos e base histórica
```

Pacote individual: `Aula 9/pacotes/controle-aquisicoes.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/controle-aquisicoes/index.html.

---

§ 27

Custódia Definitiva · Edjane Ávila

Equipe: Edjane Ávila · **Tipo:** Mapeamento e aplicativo · **Repositório:** https://github.com/renato0503/custodia-definitiva

Recebimento, endereçamento e localização de vestígios na custódia definitiva.

O QUE JÁ ESTÁ PRONTO

- Cadastro e endereço hierárquico
- Endereço sugerido x efetivo e confirmação física
- Acervo, movimentações e painel de ocupação
- Alertas de capacidade e compatibilidade

O QUE FALTA

- Confirmar a estrutura física real e as capacidades
- Definir a unidade de medida e a compatibilidade entre categorias
- Medir o tempo de recebimento e de localização

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/custodia-definitiva e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/custodia-definitiva.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\custodia-definitiva`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/custodia-definitiva` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Estrutura real

1. Levante contêiner, estante, prateleira, nível e posição de cada espaço.
2. Ajuste as capacidades e a unidade em `app.json` e no cadastro.
3. Recadastre os espaços fictícios com os reais.
4. Teste uma entrada e uma localização ponta a ponta.

Aceite

1. Escolha uma amostra de vestígios.
2. Compare o endereço sugerido com o efetivo.
3. Registre as divergências e o tempo de localização.
4. Atualize o roteiro de aceite.

Como testar

POWERSHELL

```
python construir.py
python testes/testar.py
python testes/cenarios.py
#/acervo, #/entrada e #/painel
```

Prompt para colar no Antigravity

TEXTO

```
Você está no projeto Custódia Definitiva (custodia-definitiva). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Confirmar a estrutura física real e as capacidades.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

TEXTO

```
Custódia: atualiza a estrutura física real
```

Pacote individual: `Aula 9/pacotes/custodia-definitiva.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/custodia-definitiva/index.html.

---

§ 28

DOCUMENTAPOLITEC · Aline Xavier

Equipe: Aline Xavier · **Tipo:** Aplicativo para servidor · **Repositório:** https://github.com/renato0503/documentapolitec

Controle de prontuários civis enviados ao arquivo central, com caixas, lacres e conferência.

O QUE JÁ ESTÁ PRONTO

- Caixa com relação e lacre
- Importação da planilha de 850 prontuários com associação de colunas
- Comparação com o índice digital
- Relatório de aceitas, recusadas e divergentes

O QUE FALTA

- Receber a planilha real de 850 prontuários
- Receber uma amostra do PDF digitalizado
- Confirmar o formato do prontuário e o modelo das caixas
- Implementar o QR Code se o setor mantiver o pedido

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/documentapolitec e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/documentapolitec.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\documentapolitec`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/documentapolitec` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Planilha e PDF

1. Coloque a planilha real em `exemplos/` ou importe em `#/importar`.
2. Confira o mapeamento das colunas e o relatório.
3. Teste uma amostra do PDF digitalizado, sem dado pessoal.
4. Ajuste as regras de aceite e recusa.

QR Code

1. Decida se o QR Code será impresso na caixa ou no prontuário.

2. Gere o código offline com uma biblioteca embutida em `vendor/`.
3. Inclua o código no PDF de conferência.
4. Teste a leitura com um celular.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/importar e #/relatorio-importacao
```

Prompt para colar no Antigravity

```texto
Você está no projeto DOCUMENTAPOLITEC (documentapolitec). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Receber a planilha real de 850 prontuários.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```texto
DOCUMENTAPOLITEC: valida planilha e QR Code
```

Pacote individual: `Aula 9/pacotes/documentapolitec.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/documentapolitec/index.html.

---

§ 29

Mapa de Equipamentos · João Lucas

Equipe: João Lucas · **Tipo:** Aplicativo web e mapa · **Repositório:** https://github.com/renato0503/equipamentos-google-earth

Equipamentos da POLITEC por unidade, contratos, paradas e mapa no Google Earth.

O QUE JÁ ESTÁ PRONTO

- Unidades, equipamentos, contratos e paradas
- Mapa com filtros, zoom, distritos e painel
- KMZ, KML, GeoJSON e link do Google Maps
- Gráficos, custos e privacidade

O QUE FALTA

- Testar o `.kmz` no Google Earth Pro e no Web
- Confirmar a base de diretorias, unidades e equipamentos
- Escolher as 3 unidades do piloto e medir a linha de base

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/equipamentos-google-earth e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/equipamentos-google-earth.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\equipamentos-google-earth`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/equipamentos-google-earth` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Google Earth

1. Abra o app e exporte o `.kmz` em `#/exportar`.
2. Abra no Google Earth Pro e no Web.
3. Confira os pinos, as fotos e o balão de cada equipamento.
4. Registre o resultado em `modelos/roteiro-aceite.md` e abra issue se algo falhar.

Base real

1. Importe a planilha das unidades do piloto.
2. Complete fotos e contratos de 10 equipamentos.

3. Registre uma parada e confira o custo no painel.
4. Não use CPF ou dado pessoal de servidor.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/mapa, #/exportar e #/painel
```

Prompt para colar no Antigravity

```texto
Você está no projeto Mapa de Equipamentos (equipamentos-google-earth). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Testar o `.kmz` no Google Earth Pro e no Web.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```texto
Equipamentos: registra o teste no Google Earth
```

Pacote individual: `Aula 9/pacotes/equipamentos-google-earth.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/equipamentos-google-earth/index.html.

---

§ 30

Gestão de Documentos · Vitória Araújo

Equipe: Vitória Araújo · **Tipo:** Aplicativo para servidor · **Repositório:** https://github.com/renato0503/gestao-documentos

Processos e prazos do Gabinete de Direção, com minutas, ofícios e checklist.

O QUE JÁ ESTÁ PRONTO

- Autuação com anexos e número AAAA.NNNN-SD
- Sinal de prazo e cobrança em 70% e no vencimento
- Minutas, ofício e despacho
- Checklist por etapa e anexo opcional

O QUE FALTA

- Validar a matriz de papéis e quem pode excluir
- Definir os anexos obrigatórios por tipo
- Ligar o canal de cobrança e medir a linha de base

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/gestao-documentos e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/gestao-documentos.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\gestao-documentos`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/gestao-documentos` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Papéis e anexos

1. Liste quem cria, tramita, aprova e exclui.
2. Ajuste as permissões no `app.json` e nos botões.
3. Defina os anexos obrigatórios por tipo de processo.
4. Teste cada perfil.

Cobrança

1. Decida entre painel local e e-mail institucional.
2. Mantenha a fila local com data, destinatário e texto.
3. Teste o aviso em 70% e no vencimento.
4. Registre quem recebeu e quando.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/processos, #/processo/:id e #/painel
```

Prompt para colar no Antigravity

```texto
Você está no projeto Gestão de Documentos (gestao-documentos). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Validar a matriz de papéis e quem pode excluir.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```texto
Gestão de Documentos: fecha papéis e cobrança
```

Pacote individual: `Aula 9/pacotes/gestao-documentos.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/gestao-documentos/index.html.

---

§ 31

NIHDE · Gestão de Identificados · Kesia Melo

Equipe: Kesia Melo · **Tipo:** Aplicativo web e portal · **Repositório:** https://github.com/renato0503/gestao_identificados

Corpos não identificados e não reclamados, exames, família, sepultamento e relatório.

O QUE JÁ ESTÁ PRONTO

- Cadastro, exames, identificação e família
- Sepultamento, entrega e auditoria
- Relatório e CSV
- Módulo CNPD/NamUs e cruzamento com os corpos

O QUE FALTA

- Confirmar o nome oficial do NIHDE e o escopo dos dados físicos
- Integrar com o Atena e a base de desaparecidos
- Validar os pesos do cruzamento e o fluxo de exames complementares

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/gestao_identificados e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.

2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/gestao_identificados.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\gestao_identificados`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/gestao_identificados` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Atena

1. Peça à TI o dicionário de dados e o caminho de integração.
2. Crie um importador/exportador em JSON e registre a origem.
3. Nunca misture dado público com o cadastro do corpo.
4. Teste com um caso fictício e registre no log.

Cruzamento

1. Escolha um desaparecido e clique em **Ver corpos**.
2. Confira os motivos e a lista de até 10 candidatos.
3. Confirme, descarte ou peça exame com justificativa.
4. Valide o resultado com o NIHDE antes do piloto.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/desaparecidos e #/corpo/:id
```

Prompt para colar no Antigravity

```
TEXTO
Você está no projeto NIHDE · Gestão de Identificados (gestao_identificados). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Confirmar o nome oficial do NIHDE e o escopo dos dados físicos.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```
TEXTO
NIHDE: valida o cruzamento com o setor
```

Pacote individual: `Aula 9/pacotes/gestao_identificados.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/gestao_identificados/index.html.

---

§ 32

Hórus · Banco de Demandas · Iohan

Equipe: Iohan · **Tipo:** Aplicativo web · **Repositório:** https://github.com/renato0503/horus-banco-demandas

Banco de demandas de contratações, com cards, ficha e relatório.

O QUE JÁ ESTÁ PRONTO

- Cadastro da demanda e regras R1 a R8
- Cards, filtros e ficha
- PDF e exportação
- Dados fictícios e testes

O QUE FALTA

- Decidir o papel da versão offline, substituir, complementar ou exercício
- Copiar o esquema real do Hórus e definir o formato de importação
- Definir o relatório e o caminho de volta ao Hórus

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/horus-banco-demandas e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/horus-banco-demandas.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\horus-banco-demandas`. Evite OneDrive e caminhos com muitos espaços.

4. Abra o **Antigravity**, escolha **File > Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/horus-banco-demandas` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Esquema do Hórus

1. Copie o DDL das tabelas reais para `horus/esquema.sql`.
2. Compare com o `contexto.md` e ajuste os campos.
3. Crie um arquivo `horus-import-ciclo.json` de exemplo.
4. Teste a importação em um ambiente de teste.

Relatório

1. Decida entre impressão do navegador e PDF embutido.
2. Inclua o ciclo, o órgão e as linhas das três tabelas.
3. Valide com a equipe do Hórus.
4. Registre o caminho de importação.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/demandas e #/demanda/:id
```

Prompt para colar no Antigravity

```
TEXTO
Você está no projeto Hórus · Banco de Demandas (horus-banco-demandas). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Decidir o papel da versão offline, substituir, complementar ou exercício.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```
TEXTO
Hórus: prepara importação do esquema real
```

Pacote individual: `Aula 9/pacotes/horus-banco-demandas.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/horus-banco-demandas/index.html.

---

§ 33

Inscrições em Cursos · Edilaura Rondon

Equipe: Edilaura Rondon · **Tipo:** Aplicativo para servidor · **Repositório:** https://github.com/renato0503/inscricoes-cursos

Curso, link e QR, formulário, importação de e-mails, triagem, convocação e turma.

O QUE JÁ ESTÁ PRONTO

- Curso, vagas, link e QR Code
- Formulário e comprovante
- Importação de e-mails e triagem pelas diretorias
- Convocação, desistência e reserva
- Perfis, instrutores, monitores e termo

O QUE FALTA

- Testar o formulário no Outlook desktop, Web e celular
- Decidir se o CPF sai do formulário
- Ativar a resposta automática da caixa `capacitacao@`
- Publicar no GitHub Pages e abrir o piloto

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/inscricoes-cursos e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.

2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/inscricoes-cursos.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\inscricoes-cursos`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/inscricoes-cursos` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Outlook e celular

1. Envie o link para você mesmo em três clientes.
2. Confira o formulário, o comprovante e o bloco de e-mail.
3. Corrija o que quebrar e registre a versão testada.
4. Peça o aceite da Edilaura.

Publicação

1. Ative o GitHub Pages no repositório, branch `main`, pasta raiz.
2. Abra a URL publicada e confira o link de inscrição.
3. Teste uma inscrição fictícia de ponta a ponta.
4. Não divulgue antes do aceite.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/divulgar/:curso e #/inscricao
```

Prompt para colar no Antigravity

```
TEXTO
Você está no projeto Inscrições em Cursos (inscricoes-cursos). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Testar o formulário no Outlook desktop, Web e celular.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```
TEXTO
Inscrições: publica e testa no Outlook
```

Pacote individual: `Aula 9/pacotes/inscricoes-cursos.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/inscricoes-cursos/index.html.

---

§ 34

Mapa de Distribuição · Igor Henrique

Equipe: Igor Henrique · **Tipo:** Automação de planilhas · **Repositório:** https://github.com/renato0503/mapa-distribuicao-credor

Importa as planilhas e o PDF do pregoeiro, monta o mapa por item e a distribuição orçamentária.

O QUE JÁ ESTÁ PRONTO

- Importação de `.xlsx`, `.csv` e PDF com camada de texto
- Mapa por item e por órgão
- Resultado, empresas e conferência
- Distribuição orçamentária e mapa SESP

O QUE FALTA

- Receber o `WorkFlow.docx` e o `mapa-manual-referencia.xlsx`
- Confirmar o modelo oficial do mapa SESP
- Conseguir uma amostra real do PDF do pregoeiro
- Fechar os testes contra o gabarito

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/mapa-distribuicao-credor e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.

2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/mapa-distribuicao-credor.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.

3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\mapa-distribuicao-credor`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File > Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/mapa-distribuicao-credor` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Gabarito e modelo

1. Coloque o gabarito em `modelos/` e compare item a item.
2. Ajuste o mapa orçamentário ao modelo da SESP.
3. Teste um caso de dotação insuficiente.
4. Registre as diferenças encontradas.

PDF real

1. Peça uma amostra sem dado pessoal.
2. Confira o cabeçalho, as colunas e as linhas de total.
3. Ajuste a associação de colunas se necessário.
4. Rode a conferência e compare com o gabarito.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/importar/:proc e #/exportar/:proc
```

Prompt para colar no Antigravity

```
TEXTO
Você está no projeto Mapa de Distribuição (mapa-distribuicao-credor). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Receber o `WorkFlow.docx` e o `mapa-manual-referencia.xlsx`.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```
TEXTO
Mapa: valida o gabarito e o modelo SESP
```

Pacote individual: `Aula 9/pacotes/mapa-distribuicao-credor.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/mapa-distribuicao-credor/index.html.

---

§ 35

Metas pelo BI · Karine Assunção

Equipe: Karine Assunção · **Tipo:** Automação de rotina · **Repositório:** https://github.com/renato0503/metas-bi-ocr

Importa o mês do Power BI, confere, aprova e gera a planilha.

O QUE JÁ ESTÁ PRONTO

- Leitura de CSV, Excel e PDF com texto
- Filtro de gerência e recorte de unidades monitoradas
- Conferência, correção e aprovação
- Geração da planilha

O QUE FALTA

- Confirmar se o Power BI permite exportar o visual
- Receber uma amostra do PDF mensal
- Confirmar o cabeçalho da planilha e as unidades
- Avaliar OCR de PDF em imagem

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/metas-bi-ocr e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/metas-bi-ocr.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.

3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\metas-bi-ocr`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File > Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/metas-bi-ocr` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Exportação do Power BI

1. Peça à TI a opção Exportar dados do visual.
2. Teste CSV e Excel com o mês real.
3. Ajuste o mapeamento de colunas se o cabeçalho mudar.
4. Registre a versão do Power BI usada.

OCR

1. Só implemente se o PDF em imagem for inevitável.
2. Prefira uma página `ocr.html` separada para não pesar o app.
3. Valide o número lido com confiança e correção humana.
4. Nunca aprove sem conferência.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/importar, #/conferencia/:mes e #/exportar/:mes
```

Prompt para colar no Antigravity

```
TEXTO
Você está no projeto Metas pelo BI (metas-bi-ocr). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Confirmar se o Power BI permite exportar o visual.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```
TEXTO
Metas: integra a exportação real do Power BI
```

Pacote individual: `Aula 9/pacotes/metas-bi-ocr.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/metas-bi-ocr/index.html.

---

§ 36

Passivo Ambiental · Fábio

Equipe: Fábio · **Tipo:** Mapeamento e aplicativo · **Repositório:** https://github.com/renato0503/passivo-ambiental

Laudos ambientais atrasados, com requisição, mapas, metodologia, valoração e quesitos.

O QUE JÁ ESTÁ PRONTO

- Requisição e cabeçalho do laudo
- Editor de mapas com três mapas obrigatórios
- Metodologia, considerações e valoração
- Quesitos, PDF e `.docx`

O QUE FALTA

- Receber o modelo oficial do laudo e uma requisição anonimizada
- Confirmar o texto da metodologia e os critérios de valoração
- Validar o modelo de mapa com o perito

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/passivo-ambiental e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/passivo-ambiental.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\passivo-ambiental`. Evite OneDrive e caminhos com muitos espaços.

4. Abra o **Antigravity**, escolha **File > Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/passivo-ambiental` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Modelo oficial

1. Compare o laudo gerado com o modelo da unidade.
2. Ajuste os títulos, o cabeçalho e as seções.
3. Inclua o texto padrão de metodologia quando aprovado.
4. Teste com uma requisição fictícia.

Valoração e mapas

1. Confirme a área, o valor de referência e a memória de cálculo.
2. Padronize norte, escala e legenda dos três mapas.
3. Gere o PDF e o `.docx` e confira as imagens.
4. Peça a revisão do perito.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/requisicoes e #/laudo/:id
```

Prompt para colar no Antigravity

```texto
Você está no projeto Passivo Ambiental (passivo-ambiental). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Receber o modelo oficial do laudo e uma requisição anonimizada.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os
testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```
TEXTO
Passivo: usa o modelo e a valoração oficiais
```

Pacote individual: `Aula 9/pacotes/passivo-ambiental.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/passivo-ambiental/index.html.

---

§ 37

Renovação de Contratos · Rodrigo Neves

Equipe: Rodrigo Neves · **Tipo:** Mapeamento de processo · **Repositório:** https://github.com/renato0503/renovacao-contratos-horus

Semáforo de vencimento, instrução, aditivos, fiscais e avisos de contrato.

O QUE JÁ ESTÁ PRONTO

- Cadastro e semáforo de seis faixas
- Aditivos, reajustes e instrução
- Fiscais e gestores com contato
- Edição, avisos e importação da planilha

O QUE FALTA

- Confirmar os contatos reais dos fiscais e gestores
- Definir o canal de e-mail e a confirmação de leitura
- Confirmar as faixas e o modelo da planilha do setor
- Validar as permissões de edição

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/renovacao-contratos-horus e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/renovacao-contratos-horus.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\renovacao-contratos-horus`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/renovacao-contratos-horus` para criar uma branch de trabalho.

6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Contatos e planilha

1. Preencha os fiscais e gestores no cadastro do contrato.
2. Importe a planilha do setor em `#/contratos`.
3. Confira os contratos atualizados e o semáforo.
4. Corrija a planilha modelo se necessário.

Avisos

1. Mantenha a fila local com destinatário, texto e data.
2. Para envio real, crie um Apps Script com `MailApp`.
3. Teste laranja e vermelho com contratos fictícios.
4. Registre leitura e resposta no app.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/painel, #/instrucao e #/contrato/:id
```

Prompt para colar no Antigravity

```texto
Você está no projeto Renovação de Contratos (renovacao-contratos-horus). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Confirmar os contatos reais dos fiscais e gestores.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```texto
Renovação: liga contatos e avisos reais
```

Pacote individual: `Aula 9/pacotes/renovacao-contratos-horus.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/renovacao-contratos-horus/index.html.

---

§ 38

Registro de Não Conformidade · Sabrina Ellen

Equipe: Sabrina Ellen · **Tipo:** Formulário e painel · **Repositório:** https://github.com/renato0503/rnc-nao-conformidade

RNC com importação da planilha oficial, etapas, prazos, plano de ação e documento.

O QUE JÁ ESTÁ PRONTO

- Importação do `.xlsx` oficial e do Google Planilhas
- Etapas, prazos e complemento em 5 dias úteis
- Plano de ação, avaliação e relacionamento
- Relação do setor e documento para impressão

O QUE FALTA

- Receber o modelo oficial e as categorias da CQUAL
- Definir login e permissões reais
- Validar a planilha real e o canal de e-mail

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/rnc-nao-conformidade e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/rnc-nao-conformidade.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\rnc-nao-conformidade`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/rnc-nao-conformidade` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Planilha e categorias

1. Coloque a planilha oficial em `modelos/`.
2. Ajuste as 23 colunas e as categorias em `src/app.js`.
3. Importe uma resposta fictícia e confira a ficha.
4. Valide com a CQUAL.

Login e e-mail

1. A autenticação real não fica no navegador; use o provedor institucional.
2. Mantenha a fila local para os avisos.
3. Teste a relação do setor e o PDF.
4. Registre as permissões.

Como testar

POWERSHELL

```
python construir.py
python testes/testar.py
python testes/cenarios.py
#/importar e #/rnc/:id
```

Prompt para colar no Antigravity

TEXTO

```
Você está no projeto Registro de Não Conformidade (rnc-nao-conformidade). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Receber o modelo oficial e as categorias da CQUAL.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

TEXTO

```
RNC: valida a planilha e as permissões oficiais
```

Pacote individual: `Aula 9/pacotes/rnc-nao-conformidade.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/rnc-nao-conformidade/index.html.

§ 39

TAP Digital · Danielle, Cristiane e Luciana

Equipe: Danielle, Cristiane e Luciana · **Tipo:** Formulário e portfólio · **Repositório:** https://github.com/renato0503/tap-portfolio-projetos

TAP guiado, triagem, parecer, decisão, portfólio e dashboard orçamentário.

O QUE JÁ ESTÁ PRONTO

- TAP guiado com checklist e pacote
- Triagem, pendência e devolução
- Parecer, decisão e portfólio
- Dashboard, objetivos editáveis e avisos

O QUE FALTA

- Confirmar os objetivos estratégicos oficiais
- Ligar o envio real dos avisos
- Validar o prazo de 5 dias e testar com a Danielle
- Ajustar a matriz do portfólio (P2)

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/tap-portfolio-projetos e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/tap-portfolio-projetos.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\tap-portfolio-projetos`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/tap-portfolio-projetos` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Objetivos e avisos

1. Cole a lista oficial em `#/config`.
2. Teste um TAP novo e confira as opções.
3. Crie uma pendência e confira a fila de avisos.
4. Para e-mail real, use Apps Script ou o canal institucional.

Fluxo e aceite

1. Crie um TAP de teste com anexos fictícios.
2. Devolva com pendência, sane e reenvie.
3. Parecer, decisão e portfólio.
4. Peça o aceite da Danielle.

Como testar

POWERSHELL

```
python construir.py
python testes/testar.py
python testes/cenarios.py
#/tap/novo, #/triagem e #/portfolio
```

Prompt para colar no Antigravity

TEXTO

```
Você está no projeto TAP Digital (tap-portfolio-projetos). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Confirmar os objetivos estratégicos oficiais.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

TEXTO

```
TAP: objetivos e avisos validados com o setor
```

Pacote individual: `Aula 9/pacotes/tap-portfolio-projetos.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/tap-portfolio-projetos/index.html.

---

§ 40

TEIA · Triagem de E-mails · Nelson e Helen

Equipe: Nelson e Helen · **Tipo:** Automação de rotina · **Repositório:** https://github.com/renato0503/teia-triagem-emails

Triagem de e-mails, extração de campos, divergências, lote e planilha.

O QUE JÁ ESTÁ PRONTO

- Fila, extração e conferência
- Divergências e OCR simulado
- Lote e planilha
- Campos do feedback, 36 regionais e reiteração

O QUE FALTA

- Ler e-mails e PDFs reais
- Integrar a caixa institucional
- Validar as regionais, os níveis e a regra de reiteração
- Definir o OCR real

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/teia-triagem-emails e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/teia-triagem-emails.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\teia-triagem-emails`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/teia-triagem-emails` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Caixa e e-mails

1. Peça à TI a caixa e o formato de exportação.
2. Implemente a leitura local de `.eml` / `.msg` ou a cola de texto.
3. Mantenha a fila e o protocolo `nnnnnn/aaaa`.
4. Teste com e-mails fictícios.

OCR e regionais

1. Confirme as 36 regionais e a grafia.
2. Se o PDF for imagem, avalie um OCR separado.
3. Valide a reiteração em 30 dias.
4. Registre as divergências.

Como testar

POWERSHELL

```
python construir.py
python testes/testar.py
python testes/cenarios.py
#/fila e #/lote
```

Prompt para colar no Antigravity

TEXTO

```
Você está no projeto TEIA · Triagem de E-mails (teia-triagem-emails). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Ler e-mails e PDFs reais.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

TEXTO

```
TEIA: prepara a caixa e o OCR reais
```

Pacote individual: `Aula 9/pacotes/teia-triagem-emails.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/teia-triagem-emails/index.html.

---

§ 41

TCIN · Trilha de Custódia · Silmara Bueno

Equipe: Silmara Bueno · **Tipo:** Aplicativo para servidor · **Repositório:** https://github.com/renato0503/trilha-custodia-necropsia

Trilha de custódia e identificação de necropsia, com sete etapas, perfis e assinatura.

O QUE JÁ ESTÁ PRONTO

- Sete etapas e três perfis
- Assinatura no canvas e sepultamento
- Recebimento de lacre e laudo
- Auditoria e resumo

O QUE FALTA

- Confirmar o nome CNC ou TCIN
- Validar os quatro marcos e os livros substituídos
- Medir a linha de base e alinhar com o IML
- Fazer o aceite e o piloto

Roteiro no GitHub, do zero ao pull request

1. Entre em https://github.com/renato0503/trilha-custodia-necropsia e aceite o convite de colaborador. Se ainda não recebeu, peça ao professor para enviar de novo.
2. Clique em **Code** > **Download ZIP** ou clone: `git clone https://github.com/renato0503/trilha-custodia-necropsia.git`. O ZIP serve para abrir e estudar; para enviar alterações, use o clone ou um fork.
3. Extraia em uma pasta curta, por exemplo `C:\POLITEC\trilha-custodia-necropsia`. Evite OneDrive e caminhos com muitos espaços.
4. Abra o **Antigravity**, escolha **File** > **Open Folder** e selecione a pasta que contém `app.json`, `src`, `nucleo`, `construir.py` e `testes`.
5. No terminal do Antigravity, rode `git checkout -b melhoria/trilha-custodia-necropsia` para criar uma branch de trabalho.
6. Leia `README.md`, `PREMISSAS.md` e `sprint.md`. Não edite `index.html`: ele é gerado pelo `construir.py`.
7. Edite as fontes, rode `python construir.py`, abra o `index.html` e rode os testes do app.
8. Faça `git add .`, `git commit -m "..."`, `git push origin melhoria/<tema>` e abra um Pull Request no GitHub. Peça revisão antes do merge.

Integração que falta, passo a passo

Marcos e livros

1. Reúna os quatro livros e os marcos com o IML.
2. Ajuste os campos e as etapas no `app.json`.
3. Teste um caso fictício do acolhimento à liberação.
4. Registre as diferenças.

Aceite

1. Rode o roteiro com a Silmara e o IML.
2. Confira as permissões dos três perfis.
3. Teste a assinatura e o recebimento.
4. Abra o piloto de 90 dias.

Como testar

```powershell
python construir.py
python testes/testar.py
python testes/cenarios.py
#/painel e #/caso/:id
```

Prompt para colar no Antigravity

```texto
Você está no projeto TCIN · Trilha de Custódia (trilha-custodia-necropsia). Leia README.md, PREMISSAS.md e sprint.md.
Não edite index.html. Trabalhe apenas em: Confirmar o nome CNC ou TCIN.
Antes de alterar, rode python construir.py e os testes. Depois de alterar, explique o que mudou, rode os testes de novo e mostre o diff. Não use dados reais nem peça token.
```

MENSAGEM DE COMMIT SUGERIDA

```texto
TCIN: valida marcos e aceite com o IML
```

Pacote individual: `Aula 9/pacotes/trilha-custodia-necropsia.md`. App publicado: https://renato0503.github.io/Aula_politec/Apps/trilha-custodia-necropsia/index.html.

PARTE 4

Receitas de integração

E-mail, Google Sheets, OCR, Earth, QR, Firestore e aceite.

1. E-mail e Apps Script
2. Google Sheets e CSV
3. OCR e PDF
4. Google Earth, KMZ e GeoJSON
5. QR Code e link de inscrição
6. Firestore e formulário público
7. Biometria e sistemas institucionais

§ 42

E-mail e Apps Script

Quase todo app precisa avisar alguém. No MVP, a fila é local. No uso real, o envio passa por Apps Script, Gmail institucional ou SMTP.

1. Crie um projeto em script.google.com com a conta institucional.
2. Cole a função abaixo e autorize.
3. Chame a função a partir do app por uma fila local ou por um botão.
4. Teste primeiro com o seu próprio e-mail.
5. Registre quem recebeu, quando e qual foi a resposta.

JAVASCRIPT

```
function enviarAviso(para, assunto, corpo) {
  MailApp.sendEmail({to: para, subject: assunto, body: corpo});
  return {ok: true, enviado_em: new Date().toISOString()};
}
```

NÃO COLOQUE TOKEN NO APP

O Apps Script usa a permissão da conta. Não embuta senha, token ou chave no `index.html`.

Para vaga única e concorrência, use `LockService.getScriptLock()` e grave em uma planilha. Para lembrete, crie um gatilho diário.

---

§ 43

Google Sheets e CSV

O Sheets é útil para uma fila compartilhada e para o setor conferir os dados sem abrir o app. O app continua offline e importa ou exporta CSV.

1. Defina a planilha e as colunas, com uma linha de exemplo.
2. No app, exporte CSV com BOM e ponto e vírgula.
3. No Sheets, use **Arquivo** > **Importar** e confira a acentuação.
4. Para leitura, publique a aba como CSV ou use Apps Script.
5. Nunca publique uma aba com nome, CPF ou dado de saúde.

```texto
Unidade;Laudos pendentes
IML Cuiabá;570
IML Várzea Grande;511
```

CSP

O app não pode buscar CSV em tempo de execução se a política for `connect-src 'none'`. A sincronização precisa ser por importação manual ou por Apps Script com a permissão do setor.

---

§ 44

OCR e PDF

PDF com camada de texto é fácil. PDF em imagem precisa de OCR. O OCR consome memória e não é 100% confiável.

1. Tente ler o texto com pdf.js. Se vier texto, use a tabela.
2. Se não vier, avise e permita digitar ou importar CSV.
3. Se o OCR for inevitável, coloque em uma página separada para não pesar o app.
4. Mostre a confiança de cada número e exija conferência humana.
5. Nunca aprove um mês ou um laudo sem revisão.

OCR NÃO É VERDADE

Um número lido com 80% de confiança não pode entrar na planilha sem alerta. O app deve marcar e pedir confirmação.

QUANDO

Use OCR só quando o setor não consegue exportar CSV ou Excel.

POR QUE

A exportação nativa é mais rápida e mais confiável.

EXEMPLO NA POLITEC

No Metas BI, o CSV do Power BI é o caminho preferido.

---

§ 45

Google Earth, KMZ e GeoJSON

O mapa do app é offline. Para o mapa real, exporte KMZ ou GeoJSON e abra no Google Earth, QGIS ou ArcGIS.

1. Gere o `.kmz` em Exportar.
2. Abra no Google Earth Pro e no Web.
3. Confira pinos, fotos, balões e cores.
4. Se o balão não abrir, verifique o tamanho da foto e o nome do arquivo.
5. Registre o teste no roteiro de aceite.

EARTH ENGINE

O Earth Engine é para análise de imagem de satélite, não para cadastro de equipamentos. Para o cadastro, use KMZ, GeoJSON ou a Google Maps API.

---

§ 46

QR Code e link de inscrição

O QR Code serve para o cidadão abrir o formulário sem digitar. O código precisa ser gerado offline e apontar para a URL publicada.

1. Publique o app no GitHub Pages.
2. Gere o link de inscrição com os dados do curso.
3. Gere o QR a partir do link e baixe o SVG.
4. Imprima e teste com dois celulares.
5. Se o link mudar, gere o QR de novo.

LINK LOCAL

Um link `file:///C:/...` não funciona no celular do cidadão. Publique antes de divulgar.

---

§ 47

Firestore e formulário público

O formulário da Aula 8 usa Firestore no plano Spark. Os arquivos vão como documentos em base64, com limite de 600 KB por arquivo.

1. Confira as regras em `Aula 8/firestore.rules`.

2. Ative a autenticação anônima, se o projeto exigir.
3. Teste o envio com um projeto fictício.
4. Confira a coleção e o documento.
5. Nunca envie dado pessoal de caso real.

PLANO SPARK

O Cloud Storage exige o plano Blaze. Por isso os arquivos vão no Firestore. Arquivo maior vai por e-mail ou WhatsApp, com o identificador do projeto.

---

§ 48

Biometria e sistemas institucionais

A integração com base biométrica, Atena, SINALID, CADÊ, SINESP, RIBPG e SINAB depende de credencial e termo. O app faz a parte local e deixa o passo a passo.

1. Peça à TI o contrato da API, o dicionário de dados e o ambiente de teste.
2. Crie uma fila local com protocolo, status e data.
3. Nunca grave a biometria ou o dado sensível no navegador.
4. Registre cada consulta no log, com finalidade.
5. Teste com resposta simulada antes de ligar a produção.

SEM CREDENCIAL, NÃO INVENTE

Não simule uma integração como se fosse real. O app deve dizer que a etapa está pendente e por quê.

PARTE 5

Equipe, revisão e entrega

Trabalho em paralelo, revisão e o que apresentar.

1. Papéis na continuidade
2. Revisão de código em 10 minutos
3. O aceite com o setor
4. O piloto e a linha de base
5. O que entregar no fim

§ 49

Papéis na continuidade

Um grupo pequeno precisa combinar quem faz o quê. O MVP já está pronto; agora o trabalho é de integração, teste e aceite.

Papel	Responsabilidade	Entrega
Coordenação	Fala com o setor, prioriza e marca o aceite	Ata e decisão
Desenvolvimento	Altera as fontes, constrói e testa	Pull Request
Dados	Cuida do seed, da planilha e da LGPD	Backup e dicionário
Teste	Roda os cenários e registra o que falhou	Issue e print
Documentação	Atualiza README, sprint e PREMISSAS	Commit de docs

UMA PESSOA POR BRANCH

Se duas pessoas mexem no mesmo arquivo, combine antes. Uma branch por tarefa reduz o conflito.

O grupo não precisa de todos os papéis em tempo integral. Precisa saber quem responde por cada um.

---

§ 50

Revisão de código em 10 minutos

A revisão não é para achar culpado. É para garantir que o app continua funcionando e que a mudança é compreensível.

- [ ] O Pull Request tem uma tarefa só.
- [ ] O commit não inclui dado real, token ou backup.
- [ ] O `index.html` foi gerado, não editado à mão.
- [ ] Os testes passam.
- [ ] O cenário novo cobre a regra nova.
- [ ] O README ou o sprint foi atualizado.
- [ ] A mensagem de commit explica o porquê.

COMENTÁRIO ÚTIL

Diga o que observar e por quê. “Aqui o prazo pode ficar negativo; teste o caso de hoje” ajuda mais do que “está errado”.

Se a mudança for grande, peça para dividir em duas. Pull Request pequeno é revisado em minutos; grande fica parado.

---

§ 51

O aceite com o setor

Aceite é o momento em que o setor confirma que o app resolve o problema. Não é uma demonstração; é um roteiro com critério.

1. Marque uma reunião de 40 minutos.
2. Abra o roteiro de aceite do app.
3. Use dados fictícios ou anonimizados.
4. Peça para o servidor do setor executar a tarefa, não você.
5. Anote o que travou e o tempo.
6. No fim, peça o aceite escrito, com data e responsável.

CRITÉRIO DE ACEITE

O critério já está no `sprint.md`. Se ele não estiver claro, escreva antes da reunião: “o app está aceito quando o servidor consegue fazer X sem ajuda”.

---

§ 52

O piloto e a linha de base

O piloto mede o app no uso real, com um grupo pequeno e por tempo limitado. A linha de base é o tempo ou o erro antes do app.

Momento	O que medir	Como registrar
Antes	Tempo, erros, retrabalho	Planilha ou print
Durante	Uso, dúvidas, falhas	Diário de bordo e issues
Depois	Ganho, custo, satisfação	Relatório de 1 página

QUANDO

Use o piloto para decidir, não para defender o app.

POR QUE

Se o app não reduzir o tempo ou o erro, o piloto mostra e vocês ajustam.

EXEMPLO NA POLITEC

Três unidades ou uma turma reduzida são suficientes para começar.

PILOTO SEM BASELINE

Sem o “antes”, o “depois” não prova nada. Meça uma semana antes de ligar o app.

---

§ 53

O que entregar no fim

A entrega final tem quatro provas: o app, o repositório, o aceite e o relatório do piloto.

- [ ] App publicado e abrindo no navegador.
- [ ] Repositório com README, PREMISSAS, sprint e modelos atualizados.
- [ ] Pull Requests revisados e mergeados.
- [ ] Roteiro de aceite assinado ou registrado.
- [ ] Relatório do piloto com a linha de base.
- [ ] Backup dos dados fictícios e dicionário de dados.
- [ ] Apresentação de 5 minutos com problema, solução, número e próximo passo.

A apresentação não precisa mostrar código. Precisa mostrar o problema, o antes, o depois e o que o setor ganhou.

5 MINUTOS

Um slide de problema, um de solução, um de número, um de próximos passos e um de equipe. O resto é conversa.

PARTE 6

Fechamento

Checklist, erros comuns, quiz, glossário e referências.

1. Checklist final do curso
2. Os erros que mais aparecem
3. Quiz final
4. Glossário
5. Referências e próximos passos

§ 54

Checklist final do curso

Use esta lista na véspera da entrega. Se algum item falhar, resolva antes de apresentar.

- [ ] O app abre com duplo clique e não pede internet.
- [ ] O seed carrega e o botão de backup funciona.
- [ ] O teste de estrutura passa.
- [ ] O cenário da última tarefa passa.
- [ ] O repositório está atualizado e a branch main está limpa.
- [ ] O GitHub Pages abre a URL publicada.
- [ ] O README diz o que o app faz e como rodar.
- [ ] O sprint.md está marcado até onde chegou.
- [ ] As premissas estão registradas, não escondidas.
- [ ] O aceite e o piloto têm data e responsável.

SE ALGO NÃO ESTIVER PRONTO

Diga o que falta. Um MVP honesto com pendência registrada vale mais do que uma demo que quebra.

---

§ 55

Os erros que mais aparecem

Estes erros já apareceram no curso. Reconheça e evite.

Erro	Sintoma	Correção
Editar o index.html	A alteração some no build	Editar a fonte e rodar construir.py
Misturar dado real	Nome ou CPF no seed	Anonimizar e refazer o seed
Não rodar o build	A tela nova não aparece	Rodar construir.py
Commit gigante	Revisão impossível	Dividir por tarefa
Teste ajustado para passar	A regra continua errada	Corrigir a regra e o cenário

Erro	Sintoma	Correção
Link local no QR	Não abre no celular	Publicar no Pages
Token no código	Vazamento	Remover do histórico e avisar
Sem backup	Dados perdidos	Exportar .json antes da reunião

O ERRO MAIS CARO

É acreditar que o teste cobre tudo. Teste cobre o que foi escrito. O aceite com o setor cobre o resto.

§ 56

Quiz final

Qual arquivo você edita para mudar uma tela?

a. index.html

b. src/app.js ou src/sprintN.js

c. app.json apenas

d. O PDF

RESPOSTA **(b) src/app.js ou src/sprintN.js.** O index.html é gerado. A tela está em src/.

Qual comando gera o index.html?

a. python testes/testar.py

b. python construir.py

c. git push

d. python validar.py

RESPOSTA **(b) python construir.py.**

O que fazer antes de abrir um Pull Request?

a. Rodar os testes e revisar o diff

b. Apagar a branch main

c. Subir o backup real

d. Editar o index.html

RESPOSTA **(a) Rodar os testes e revisar o diff.**

Onde fica o passo a passo de integração externa?

a. No README apenas

b. Na Ajuda Técnica, três cliques no nome do app

c. No index.html

d. No arquivo .pbix

RESPOSTA **(b) Na Ajuda Técnica, três cliques no nome do app.**

O que é a linha de base?

a. O tempo ou o erro antes do app

b. O nome do repositório

c. O backup do banco

d. A branch principal

RESPOSTA **(a) O tempo ou o erro antes do app.**

Qual é o caminho correto para uma alteração?

a. main, editar, push

b. branch, editar, construir, testar, commit, push, PR

c. ZIP, editar, e-mail

d. index.html, print, WhatsApp

RESPOSTA **(b) branch, editar, construir, testar, commit, push, PR.**

O que nunca pode entrar no repositório?

a. README

b. Dados fictícios

c. Token, senha e dado pessoal real

d. Sprint.md

RESPOSTA **(c) Token, senha e dado pessoal real.**

O que o aceite precisa ter?

a. Um print bonito

b. Critério, data e responsável

c. Só a opinião do professor

d. O código inteiro

RESPOSTA **(b) Critério, data e responsável.**

---

§ 57

Glossário

MVP
Menor versão do produto que resolve o problema principal e pode ser testada.

Repositório
Pasta versionada no GitHub, com histórico de commits.

Branch
Linha paralela de trabalho, usada para isolar uma alteração.

Commit
Registro de uma alteração, com autor, data e mensagem.

Pull Request
Pedido para juntar uma branch na main, com revisão.

Issue
Tarefa, erro ou dúvida registrada no GitHub.

Build
Processo que gera o index.html a partir das fontes.

Seed
Dados fictícios com semente fixa, usados nos testes e na demo.

IndexedDB
Banco do navegador onde o app guarda os dados locais.

Offline first
O app funciona sem internet e não faz requisição de rede.

Aceite
Confirmação do setor de que o app atende ao critério combinado.

Piloto
Uso real, limitado e medido, antes da implantação.

Linha de base
Medida do processo antes do app, para comparar depois.

LGPD
Lei de proteção de dados: finalidade, acesso, retenção e segurança.

---

§ 58

Referências e próximos passos

O material do curso continua no repositório. Os links principais estão abaixo.

- Repositório do curso
- Site do curso
- Hub dos 21 apps
- CONTINUAR.md
- PREMISSAS.md
- Documentação do GitHub
- Antigravity

PRÓXIMO PASSO
Escolha uma pendência do seu pacote, abra uma issue, crie a branch e faça o primeiro Pull Request. A continuidade começa com uma alteração pequena e testada.

Bom trabalho. O app já funciona; agora ele ganha o uso real, com o setor, com número e com responsabilidade.